AAMEENA I(192511284)

ASSESSMENT TOOL-5

1)Apply relational algebra operations (σ, π, ∪, -, ×, ⋈) on the given

Employee and Department relations to retrieve required records.

(σ)

```
mysql> SELECT *
    -> FROM employee
    -> WHERE salary > 50000;
+-------+---------+--------+-------+
| empno | empname | salary | depno |
+-------+---------+--------+-------+
|   102 | Ravi    |  60000 | D002  |
+-------+---------+--------+-------+
1 row in set (0.05 sec)
```

(π)

```
mysql> SELECT empname
    -> FROM employee;
+---------+
| empname |
+---------+
| Asha    |
| Ravi    |
| John    |
+---------+
3 rows in set (0.00 sec)
```

(∪)

```
mysql> SELECT empno, empname
    -> FROM employee
    -> WHERE salary >= 50000
    ->
    -> UNION
    ->
    -> SELECT empno, empname
    -> FROM employee
    -> WHERE depno = 'D001';
+-------+---------+
| empno | empname |
+-------+---------+
|   101 | Asha    |
|   102 | Ravi    |
|   103 | John    |
+-------+---------+
3 rows in set (0.05 sec)
```

(-)

```
mysql> SELECT *
    -> FROM employee
    -> WHERE empno NOT IN
    -> (
    ->     SELECT empno
    ->     FROM employee
    ->     WHERE salary < 50000
    -> );
+-------+---------+--------+-------+
| empno | empname | salary | depno |
+-------+---------+--------+-------+
|   101 | Asha    |  50000 | D001  |
|   102 | Ravi    |  60000 | D002  |
+-------+---------+--------+-------+
2 rows in set (0.01 sec)
```

(×)

```
mysql> SELECT *
    -> FROM employee
    -> CROSS JOIN dept;
+-------+---------+--------+-------+-------+---------+---------+---------+
| empno | empname | salary | depno | depno | depname | dephead | deptHOD |
+-------+---------+--------+-------+-------+---------+---------+---------+
|   103 | John    |  45000 | D001  | 101   | cse     | ke      | raj     |
|   102 | Ravi    |  60000 | D002  | 101   | cse     | ke      | raj     |
|   101 | Asha    |  50000 | D001  | 101   | cse     | ke      | raj     |
|   103 | John    |  45000 | D001  | 102   | ECE     | vi      | riya    |
|   102 | Ravi    |  60000 | D002  | 102   | ECE     | vi      | riya    |
|   101 | Asha    |  50000 | D001  | 102   | ECE     | vi      | riya    |
|   103 | John    |  45000 | D001  | 103   | EEE     | ka      | divi    |
|   102 | Ravi    |  60000 | D002  | 103   | EEE     | ka      | divi    |
|   101 | Asha    |  50000 | D001  | 103   | EEE     | ka      | divi    |
|   103 | John    |  45000 | D001  | 104   | IT      | di      | sindhu  |
|   102 | Ravi    |  60000 | D002  | 104   | IT      | di      | sindhu  |
|   101 | Asha    |  50000 | D001  | 104   | IT      | di      | sindhu  |
+-------+---------+--------+-------+-------+---------+---------+---------+
12 rows in set (0.00 sec)
```

(⋈)

```
mysql> SELECT employee.empname,
    ->        employee.salary,
    ->        dept.depname
    -> FROM employee
    -> INNER JOIN dept
    -> ON employee.depno = dept.depno;
Empty set (0.05 sec)

mysql> SELECT * FROM dept;
+-------+---------+---------+---------+
| depno | depname | dephead | deptHOD |
+-------+---------+---------+---------+
| 101   | cse     | ke      | raj     |
| 102   | ECE     | vi      | riya    |
| 103   | EEE     | ka      | divi    |
| 104   | IT      | di      | sindhu  |
+-------+---------+---------+---------+
4 rows in set (0.00 sec)
```

2) Write SQL DDL statements to create STUDENT(SID, Name, DOB, DeptID) and DEPARTMENT(DeptID, DeptName) tables with all appropriate constraints.

```
mysql> desc department;
+----------+-------------+------+-----+---------+-------+
| Field    | Type        | Null | Key | Default | Extra |
+----------+-------------+------+-----+---------+-------+
| DeptID   | int         | NO   | PRI | NULL    |       |
| DeptName | varchar(50) | NO   | UNI | NULL    |       |
+----------+-------------+------+-----+---------+-------+
2 rows in set (0.01 sec)
```

```
mysql> desc student;
+--------+-------------+------+-----+---------+-------+
| Field  | Type        | Null | Key | Default | Extra |
+--------+-------------+------+-----+---------+-------+
| SID    | int         | NO   | PRI | NULL    |       |
| Name   | varchar(50) | NO   |     | NULL    |       |
| DOB    | date        | NO   |     | NULL    |       |
| DeptID | int         | YES  | MUL | NULL    |       |
+--------+-------------+------+-----+---------+-------+
4 rows in set (0.00 sec)
```

Q3) Formulate SQL queries using GROUP BY, HAVING, and aggregate functions (COUNT, SUM, AVG, MAX, MIN) on a given Sales dataset.

```
mysql> SELECT Product, COUNT(*) AS TotalSales
    -> FROM sales
    -> GROUP BY Product;
+---------+------------+
| Product | TotalSales |
+---------+------------+
| Laptop  |          2 |
| Mobile  |          2 |
| Tablet  |          1 |
+---------+------------+
3 rows in set (0.05 sec)

mysql> SELECT Product, SUM(Amount) AS TotalAmount
    -> FROM sales
    -> GROUP BY Product;
+---------+-------------+
| Product | TotalAmount |
+---------+-------------+
| Laptop  |      110000 |
| Mobile  |       45000 |
| Tablet  |       30000 |
+---------+-------------+
3 rows in set (0.00 sec)

mysql> SELECT Product, AVG(Amount) AS AverageAmount
    -> FROM sales
    -> GROUP BY Product;
+---------+---------------+
| Product | AverageAmount |
+---------+---------------+
| Laptop  |    55000.0000 |
| Mobile  |    22500.0000 |
| Tablet  |    30000.0000 |
+---------+---------------+
3 rows in set (0.00 sec)

mysql> SELECT MAX(Amount) AS MaximumAmount
    -> FROM sales;
+---------------+
| MaximumAmount |
+---------------+
|         60000 |
+---------------+
1 row in set (0.00 sec)

mysql> SELECT MIN(Amount) AS MinimumAmount
    -> FROM sales;
+---------------+
| MinimumAmount |
+---------------+
|         20000 |
+---------------+
1 row in set (0.00 sec)

mysql> SELECT Product, SUM(Amount) AS TotalAmount
    -> FROM sales
    -> GROUP BY Product
    -> HAVING SUM(Amount) > 40000;
+---------+-------------+
| Product | TotalAmount |
+---------+-------------+
| Laptop  |      110000 |
| Mobile  |       45000 |
+---------+-------------+
2 rows in set (0.05 sec)
```

Q4) Write a correlated sub-query to find employees earning more than the average salary of their own department.

```
mysql> use dept;
Database changed
mysql> select*from employee;
+-------+---------+--------+-------+
| empno | empname | salary | depno |
+-------+---------+--------+-------+
|   101 | Asha    |  50000 | D001  |
|   102 | Ravi    |  60000 | D002  |
|   103 | John    |  45000 | D001  |
+-------+---------+--------+-------+
3 rows in set (0.00 sec)

mysql> SELECT *
    -> FROM employee E
    -> WHERE salary >
    -> (
    ->     SELECT AVG(salary)
    ->     FROM employee
    ->     WHERE depno = E.depno
    -> );
+-------+---------+--------+-------+
| empno | empname | salary | depno |
+-------+---------+--------+-------+
|   101 | Asha    |  50000 | D001  |
+-------+---------+--------+-------+
1 row in set (0.05 sec)
```

Q5)

Apply all types of JOIN operations (INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS) on Employee and Project tables.

```
mysql> SELECT employee.empno,
    ->        employee.empname,
    ->        project.projectname
    -> FROM employee
    -> INNER JOIN project
    -> ON employee.empno = project.empno;
+-------+---------+-----------------+
| empno | empname | projectname     |
+-------+---------+-----------------+
|   101 | Asha    | Website         |
|   102 | Ravi    | Billing System  |
|   103 | John    | Mobile App      |
+-------+---------+-----------------+
3 rows in set (0.00 sec)

mysql> SELECT employee.empno,
    ->        employee.empname,
    ->        project.projectname
    -> FROM employee
    -> LEFT JOIN project
    -> ON employee.empno = project.empno;
+-------+---------+-----------------+
| empno | empname | projectname     |
+-------+---------+-----------------+
|   101 | Asha    | Website         |
|   102 | Ravi    | Billing System  |
|   103 | John    | Mobile App      |
+-------+---------+-----------------+
3 rows in set (0.00 sec)

mysql> SELECT employee.empno,
    ->        employee.empname,
    ->        project.projectname
    -> FROM employee
    -> RIGHT JOIN project
    -> ON employee.empno = project.empno;
+-------+---------+-----------------+
| empno | empname | projectname     |
+-------+---------+-----------------+
|   101 | Asha    | Website         |
|   102 | Ravi    | Billing System  |
|   103 | John    | Mobile App      |
|  NULL | NULL    | AI Project      |
+-------+---------+-----------------+
4 rows in set (0.00 sec)
```

Q6) Create a view 'DeptSalaryView' that displays department name,

total employees, and average salary. Write a query on this view.

```
mysql> CREATE VIEW DeptSalaryView AS
    -> SELECT
    ->     d.depname AS DepartmentName,
    ->     COUNT(e.empno) AS TotalEmployees,
    ->     AVG(e.salary) AS AverageSalary
    -> FROM dept d
    -> INNER JOIN employee e
    -> ON d.depno = e.depno
    -> GROUP BY d.depno, d.depname;
Query OK, 0 rows affected (0.09 sec)

mysql> SELECT * FROM DeptSalaryView;
+-----------------+----------------+----------------+
| DepartmentName  | TotalEmployees | AverageSalary  |
+-----------------+----------------+----------------+
| cse             |              2 |    47500.0000  |
| ECE             |              1 |    60000.0000  |
+-----------------+----------------+----------------+
2 rows in set (0.00 sec)
```

Q7) Construct a relational algebra expression for the following: Find

names of students who are enrolled in all courses offered.

```
mysql> SELECT s.Name
    -> FROM student s
    -> JOIN enrollment e
    -> ON s.SID = e.SID
    -> GROUP BY s.SID, s.Name
    -> HAVING COUNT(DISTINCT e.CourseID) =
    -> (
    ->     SELECT COUNT(*)
    ->     FROM course
    -> );
+-------+
| Name |
+-------+
| Asha |
+-------+
1 row in set (0.08 sec)
```

Q8) Write SQL DML statements (INSERT, UPDATE, DELETE) with integrity constraint enforcement for a given scenario.

```
mysql> INSERT INTO student
    -> VALUES (104, 'Anitha', '2005-08-15', 1);
Query OK, 1 row affected (0.05 sec)

mysql> SELECT * FROM student;
+-----+--------+------------+--------+
| SID | Name   | DOB        | DeptID |
+-----+--------+------------+--------+
| 101 | Asha   | 2005-01-10 |      1 |
| 102 | Ravi   | 2005-03-15 |      1 |
| 103 | John   | 2005-07-20 |      1 |
| 104 | Anitha | 2005-08-15 |      1 |
+-----+--------+------------+--------+
4 rows in set (0.00 sec)

mysql> UPDATE student
    -> SET Name = 'Anitha R'
    -> WHERE SID = 104;
Query OK, 1 row affected (0.05 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT * FROM student
    -> WHERE SID = 104;
+-----+----------+------------+--------+
| SID | Name     | DOB        | DeptID |
+-----+----------+------------+--------+
| 104 | Anitha R | 2005-08-15 |      1 |
+-----+----------+------------+--------+
1 row in set (0.00 sec)

mysql> DELETE FROM student
    -> WHERE SID = 104;
Query OK, 1 row affected (0.05 sec)

mysql> SELECT * FROM student
    -> WHERE SID = 104;
Empty set (0.00 sec)
```

Q9) Apply tuple relational calculus to express a query: Find all

employees who work in the 'IT' department and earn more than

50000.

```
mysql> SELECT e.*
    -> FROM employee e
    -> JOIN dept d
    -> ON e.depno = d.depno
    -> WHERE d.depname = 'IT'
    -> AND e.salary > 50000;
+-------+---------+--------+-------+
| empno | empname | salary | depno |
+-------+---------+--------+-------+
|   104 | Priya   |  65000 | 104   |
+-------+---------+--------+-------+
1 row in set (0.00 sec)
```

Q10) Design a relational schema for an Airline Reservation System and

write 5 SQL queries demonstrating joins, sub-queries, and views.

```
mysql> SELECT p.PassengerName,
    ->        f.FlightName,
    ->        f.Source,
    ->        f.Destination
    -> FROM passenger p
    -> JOIN booking b
    -> ON p.PassengerID = b.PassengerID
    -> JOIN flight f
    -> ON b.FlightID = f.FlightID;
+---------------+------------+-----------+-------------+
| PassengerName | FlightName | Source    | Destination |
+---------------+------------+-----------+-------------+
| Asha          | Air India  | Chennai   | Delhi       |
| Asha          | SpiceJet   | Bengaluru | Delhi       |
| Ravi          | IndiGo     | Chennai   | Mumbai      |
+---------------+------------+-----------+-------------+
3 rows in set (0.00 sec)

mysql> SELECT PassengerName
    -> FROM passenger
    -> WHERE PassengerID IN
    -> (
    ->     SELECT PassengerID
    ->     FROM booking
    ->     GROUP BY PassengerID
    ->     HAVING COUNT(*) > 1
    -> );
+---------------+
| PassengerName |
+---------------+
| Asha          |
+---------------+
1 row in set (0.00 sec)

mysql> SELECT FlightID,
    ->        COUNT(*) AS TotalBookings
    -> FROM booking
    -> GROUP BY FlightID;
+----------+---------------+
| FlightID | TotalBookings |
+----------+---------------+
|      101 |             1 |
|      102 |             1 |
|      103 |             1 |
+----------+---------------+
3 rows in set (0.00 sec)

mysql> CREATE VIEW FlightBookingView AS
    -> SELECT f.FlightName,
    ->        COUNT(b.BookingID) AS TotalBookings
    -> FROM flight f
    -> LEFT JOIN booking b
    -> ON f.FlightID = b.FlightID
    -> GROUP BY f.FlightID, f.FlightName;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT *
    -> FROM FlightBookingView;
+------------+---------------+
| FlightName | TotalBookings |
+------------+---------------+
| Air India  |             1 |
| IndiGo     |             1 |
| SpiceJet   |             1 |
+------------+---------------+
```